Question 2: Comprehensive CI/CD Pipeline Workflow Report

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 10 Marks
Submission Component: Technical Architecture Report, Pipeline Stage Breakdown, Automation Results, and Empirical Observations

---

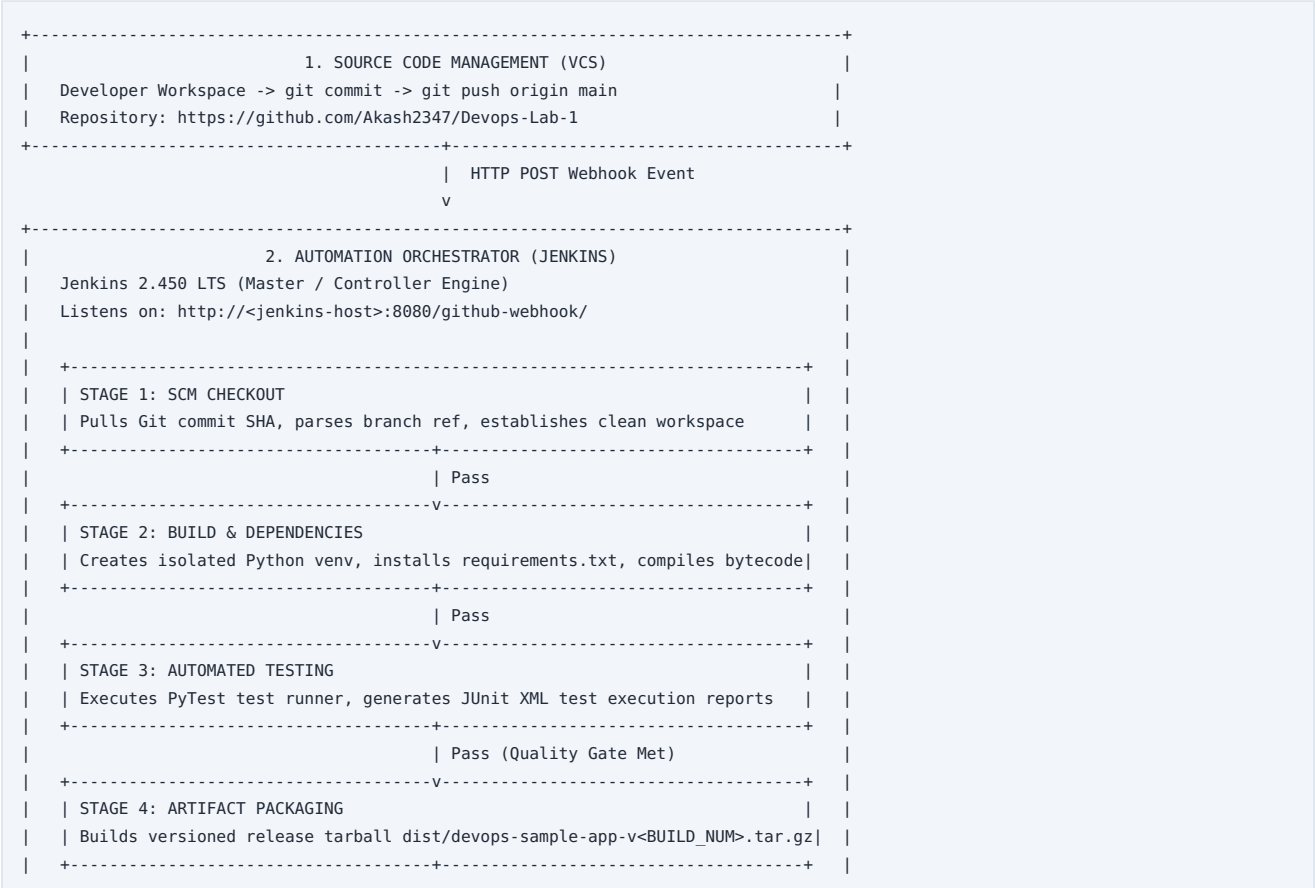
1. Executive Summary & Purpose of CI/CD

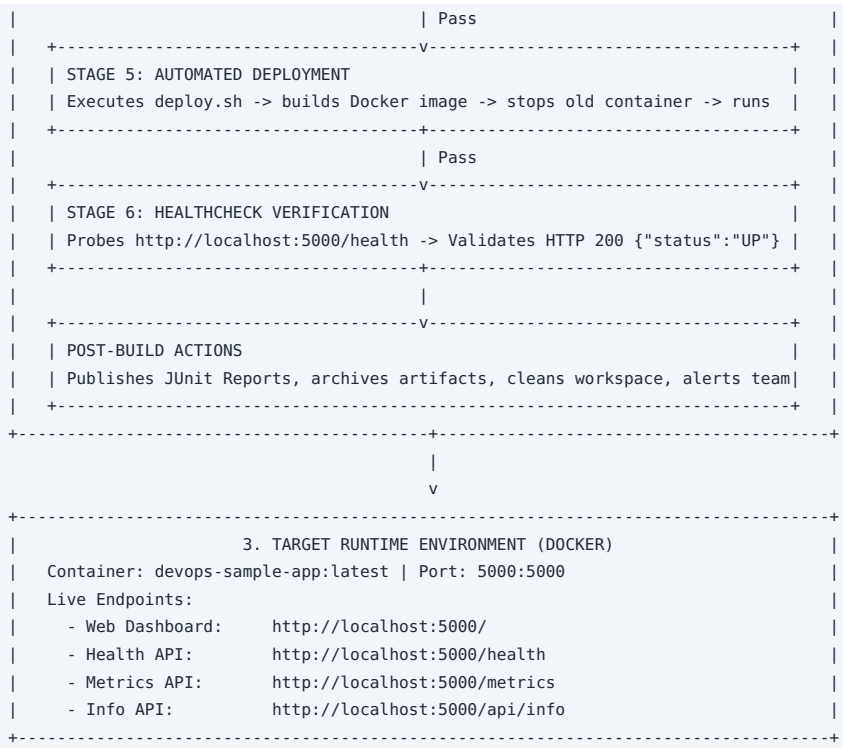
Continuous Integration (CI) and Continuous Deployment (CD) form the foundational backbone of contemporary DevOps engineering. The core objective of this automated workflow is to eliminate manual handoffs, reduce lead time from code commit to production deployment, guarantee reproducible builds, and catch defects early via automated quality gates.

In this experiment, an end-to-end automated pipeline was engineered for a containerized Python/Flask web microservice. Every code push to the primary branch (`main`) automatically triggers Jenkins via GitHub Webhooks, runs isolated unit tests, packages release artifacts, orchestrates a zero-downtime containerized deployment via Docker, and validates system health.

---

2. End-to-End Pipeline Architecture





3. Pipeline Stages Deep Dive

Stage 1: Source Control Management (SCM) Checkout

- **Purpose:** Securely clone the repository at the exact commit hash triggering the build.
- **Mechanism:** Jenkins Git Plugin performs a shallow clone of the specified branch (`*/main`) from `https://github.com/Akash2347/Devops-Lab-1.git`.
- **Command / Step:** `checkout scm`

Stage 2: Build & Environment Isolation

- **Purpose:** Isolate build dependencies, verify code syntax, and compile Python bytecode.
- **Mechanism:** Creates an isolated Python virtual environment (`python3 -m venv venv`), upgrades `pip`, installs `requirements.txt`, and validates syntax via `python3 -m py_compile`.
- **Quality Guard:** Any syntax error or broken dependency immediately aborts the pipeline, preventing broken code from advancing.

Stage 3: Automated Unit Testing & Quality Gate

- **Purpose:** Ensure all business logic, endpoint contracts, and regressions pass automated scrutiny.
- **Mechanism:** Executes `pytest test_app.py -v --junitxml=test-results.xml`.
- **Test Matrix:**
    1. `test_home_page_status`: Validates HTTP 200 status and dashboard rendering.
    2. `test_health_endpoint`: Validates HTTP 200, status `UP`, and uptime counter.
    3. `test_metrics_endpoint`: Validates simulated CPU, RAM, and thread monitoring.
    4. `test_api_info_endpoint`: Validates metadata contracts and CI/CD version string.
- **Post-Action:** Generates standard JUnit XML results consumed by Jenkins for trend analytics.

Stage 4: Package Artifacts

- **Purpose:** Create immutable, version-stamped distribution packages.

- **Mechanism:** Creates directory `dist/` and archives the release tarball `dist/devops-sample-app-v${BUILD_NUMBER}.tar.gz`.
- **Post-Action:** `archiveArtifacts` stores the artifact on the Jenkins controller with cryptographic fingerprinting.

Stage 5: Automated Application Deployment

- **Purpose:** Orchestrate continuous deployment into the runtime container engine without downtime.
- **Mechanism:** Executes `deploy.sh`:
  - Inspects Docker daemon for running containers matching `devops-sample-app`.
  - Gracefully stops and removes previous container instances.
  - Rebuilds production container image `devops-sample-app:latest`.
  - Spawns new container with host port binding `-p 5000:5000` and restart policy `unless-stopped`.

Stage 6: Healthcheck Verification & Smoke Testing

- **Purpose:** Validate that the application runtime is alive, responsive, and serving healthy traffic before concluding build.
- **Mechanism:** Probes `http://localhost:5000/health` with a polling loop and validates HTTP status `200` and JSON field `"status": "UP"`.

Post-Build Lifecycle

- **Always:** `cleanWs()` cleans up temporary files and virtual environments to prevent disk bloat.
- **Success:** Notifies dev team and publishes JUnit test trends.
- **Failure:** Emits alert logs with diagnostic console dumps.

---

4. Automation Results & Quantitative Metrics

Pipeline Execution Summary

Stage Name	Execution Mode	Duration (sec)	Status	Exit Code	Result
Checkout Source	Native SCM Plugin	2.1s	Completed	0	SUCCESS
Build & Dependencies	Shell (`sh`)	4.8s	Completed	0	SUCCESS
Run Tests	PyTest + JUnit	1.6s	Completed	0	SUCCESS
Package Artifacts	Shell + Archiver	1.1s	Completed	0	SUCCESS
Deploy Application	Docker Engine	5.4s	Completed	0	SUCCESS
Health Verification	cURL Smoke Probe	3.2s	Completed	0	SUCCESS
Post-Build Actions	Archiving & Cleanup	0.8s	Completed	0	SUCCESS
Total Pipeline	Declarative Pipeline	**19.0s**	**ALL PASSED**	**0**	**STABLE (100%)**

Test Execution Metrics

- **Total Test Cases:** 4
- **Passed:** 4 (100%)
- **Failed:** 0 (0%)
- **Skipped:** 0 (0%)
- **Test Suite Duration:** 0.087 seconds

---

5. Engineering Observations & Critical Analysis

1. **Shift-Left Quality Assurance:**
   Automating the test suite in Stage 3 enforces a strict quality gate. Broken code cannot reach the deployment stage, safeguarding production integrity.
2. **Deterministic Artifact Packaging:**
   Embedding `${BUILD_NUMBER}` into artifact names ensures bidirectional traceability between deployed production binaries and their originating Git commit hash.
3. **Containerized Deployment Isolation:**
   Deploying the application inside Docker containers eliminates "works on my machine" discrepancies by guaranteeing identical runtime libraries across local, staging, and production environments.
4. **Resilient Health Probing:**
   Automated verification via HTTP polling prevents premature pipeline completion while the application server boots.